# The Architect

**Assignment #5, Goal-Oriented Coding Agent. Rex Machina capstone.**

An agent that reads the Rex Machina GDD, scans the game's source, scores what is missing, and writes one feature into the running game.

Run the agent with `python architect/architect.py`. Play the result with `python game/play.py`. Neither needs an API key, a network connection, or a single third-party package.

---

## 1. What it built

`self_charge_pct`, the robot's solar charge and battery drain, plus the register shift that hangs off it.

Before the run, the Nemesis had a method called `charge_band` whose entire body was `return "clinical"`. The Assignment #4 pipeline had already generated 24 read lines keyed across three registers, clinical, confident and strained, and the game could only ever reach eight of them. The other sixteen were unreachable content sitting in a DataTable.

After the run, charge starts at 100, a pursuing round spends 8 and recovers 2 by solar, and a stationary round recovers 1 net. The register crosses into confident around round 7 of 15 and into strained on a fight that drags. The `charge_strain` read category, which GDD §4 lists and which had no way to fire, now fires.

The agent changed one file, `game/rex/nemesis.py`.

## 2. Why it picked that feature

It scored it highest, on the four factors Class 7 slide 11 names. The weights are a design decision and the run prints them so a reader can disagree:

```
[5] UTILITY SCORING   weights {'dependency': 0.3, 'blocking': 0.3, 'priority': 0.25, 'state': 0.15}
    self_charge_pct     dependency 1.00 x 0.30  blocking 1.00 x 0.30  priority 1.00 x 0.25  state
0.40 x 0.15  =  0.9100
    aggression          dependency 0.00 x 0.30  blocking 0.33 x 0.30  priority 0.75 x 0.25  state
0.40 x 0.15  =  0.3475
    charge_band_reads   dependency 0.00 x 0.30  blocking 0.00 x 0.30  priority 0.75 x 0.25  state
1.00 x 0.15  =  0.3375
    intercept_move      dependency 0.00 x 0.30  blocking 0.00 x 0.30  priority 1.00 x 0.25  state
0.40 x 0.15  =  0.3100
    sprint_move         dependency 0.00 x 0.30  blocking 0.00 x 0.30  priority 1.00 x 0.25  state
0.40 x 0.15  =  0.3100
    checkpoint_state    dependency 0.00 x 0.30  blocking 0.33 x 0.30  priority 0.50 x 0.25  state
0.40 x 0.15  =  0.2850
    prior_attempt_read  dependency 0.00 x 0.30  blocking 0.00 x 0.30  priority 0.50 x 0.25  state
0.40 x 0.15  =  0.1850
```

The gap between 0.91 and 0.35 is dependency order doing its job. `self_charge_pct` is the only open feature with no unmet prerequisites, and three others sit behind it: the register shift needs charge to read, aggression needs charge to spend, and the two-tile sprint needs both. Every rival scored 0.00 on dependency because each was waiting on something that did not exist yet. That is the shop UI and the inventory system from the lecture example, in this game's vocabulary.

Priority came from the GDD rather than from me. `self_charge_pct` is a field in the Nemesis input contract in §4 and "the machine tires" is one of the two levers §3 names as the player's counterplay, so it is marked critical.

## 3. Did it run in the game

Yes. `python game/play.py` and the charge appears on the HUD every round:

```
  phase 3/3  train yard  round  8/15  stamina  7  approach 0.73
  robot charge  52%  register confident
    .     .     .     .    exit   .     .     .     .     .
    .     .     .     .     .     .     .    girl   .     .
    .     .     .     .     .     .     .     .     .     .
    .     .     .     .     .     .     .    REX    .     .
    .     .   cover   .     .     .     .    kid    .     .
  (five more rows)
  girl is you. REX is the robot. kid is where you are going.
```

The grid spells the actors out rather than using initials. The story bible in `docs/` never names the dog and records that the kid called her girl, so the board uses that word, and the robot is a REX-line unit. `cover` and `exit` are dressing carried in from the Assignment 4 arena table. Neither blocks a move in this slice, and the board says so instead of implying otherwise.

A clean line still wins in 11 rounds with 1 stamina to spare. A wandering line still loses. 35 assertions in `tests/test_architect.py` pass, including the one that matters most, which is that the fight remains winnable after the agent touches it.

If you would rather not run anything, `PLAYTHROUGH.md` is the same evidence on paper, and it is generated by `tools/record_playthrough.py` rather than typed. It carries a round by round table of the winning line with the charge and register columns, and then the same game played twice with one constant changed, which is the argument in section 4 shown rather than asserted. Charge falls 94, 88, 82, 76, 70, 64, 58 and the register turns at round 7. Under the constants the agent first proposed it goes 98, 96, 94, 92, 90, 88, 86 and never turns at all.

## 4. What the agent got wrong, and what I changed

This is the part worth reading. The verifier passed the generated code and the generated code was still wrong.

**The feature did nothing.** The agent proposed a pursuit cost of 4 against a solar recovery of 2, a net drain of 2 a round. Across a 15 round fight that moves charge from 100 to about 76, which never reaches the 60 threshold, so the register never left clinical. Import passed. Tests passed. The feature was, in play, invisible. I raised the pursuit cost to 8 so the register crosses both thresholds inside one fight, and left a comment in the file recording that the agent chose 4 and why 8 replaced it. The numbers are still traceable to the GDD: §4's worked contract shows `self_charge_pct: 48`, and 8 out with 2 back reaches 48 at round 9 of 15.

**It stored the value under a name the GDD does not use.** The agent kept charge on `self.charge` and never exposed `self_charge_pct`, which is what the §4 input contract actually calls the field. That is a real defect and it had a visible consequence: on the next run the agent's own scanner could not find the feature it had just written, and reported it as "mentioned" rather than "implemented". It was skipped only because BUILT.md remembered it. I added a `self_charge_pct` property. The scanner now reports it correctly.

**The tell was buried.** GDD §3 treats the register shift as a player-facing tell, but the only channel for it was the read line, and §4's suppression rule drops any line whose category repeats. On a straight run that ate most of them, so the feature was invisible even once tuned. I put charge and register on the HUD.

Two bugs in my own agent surfaced the same way, and both are kept as regression tests. The gap detector originally excluded exact name matches, which reported five working systems as absent and would have sent the agent to rebuild code that already worked. It also ignored attributes entirely, so the engine-side intercept veto read as missing when `rejected_intercepts` was sitting in the constructor. A detector that over-fires is not cautious, it just sends the agent to rewrite things that already work.

## 5. How the agent works

Raw orchestration, per Class 7 slide 21. Two model turns, each a single HTTPS call in `architect/llm.py` written with `urllib`. Everything between the turns is plain Python a reviewer can read.

```
docs/gdd_rex_machina.md
        |
        v
  [1] gdd.py         109 candidate requirements, deterministic
        |            39 from JSON agent contracts, 70 from bolded prose
        v
  [2] scan.py        7 modules, 357 symbols, 1 constant-returning stub
        |            ast, not regex, so a comment is not mistaken for code
        v
  [3] MODEL TURN     candidates -> 16 runtime systems with a dependency graph
        |
        v
  [4] gaps.py        implemented | stubbed | mentioned | absent
        |
        v
  [5] score.py       dependency, blocking, priority, state
        |
        v
  [6] select         skips anything in BUILT.md or FAILED.md
        |
        v
  [7] MODEL TURN     writes the file, given the GDD evidence and current source
        |
        v
  [8] verify         import, then the suite. Failure rolls the tree back.
        |
        v
  state/*.md         BUILT, FAILED, NEXT, DECISIONS
```

Three details are load-bearing. The scanner distinguishes a function that carries behaviour from one whose body is a single literal return, which is how `charge_band` was caught. Generation happens against a copy, and a failed verification restores the tree, so a bad run cannot leave the game broken. Selection consults markdown memory first, which is what makes a second run advance rather than repeat: run two skips `self_charge_pct` and selects `aggression`, whose prerequisite now exists.

## 6. Provenance, stated plainly

There is no live Haiku 4.5 run in this submission. No API key was available in the environment where it was built.

The two model turns in `transcript/` were answered by Claude Opus 5 working in the authoring session, recorded under a sha256 of the exact prompt the pipeline builds. `ReplayProvider` will refuse to serve a turn whose prompt has changed by a single character, so a replay cannot drift from the prompt that produced it.

Everything except the language half is real code doing real work: the GDD parse, the ast scan, gap classification, the scoring arithmetic, the rollback, the verification, the markdown memory. `python architect/architect.py --live` with `ANTHROPIC_API_KEY` set makes the same two calls against `claude-haiku-4-5` and records the replies in place.

The live path is not merely written, it has been exercised. Running it with a deliberately invalid key produces this, and leaves the game untouched:

```
Live call failed, HTTP 401.
invalid x-api-key
The key was rejected. Check it was pasted whole and has not been revoked.
```

The three failures worth naming are handled separately: a rejected key, an empty credit balance, and a rate limit each get their own message, because a Max subscription covers Claude.ai and Claude Code but not the API, and that is the error most likely to appear first. `run_live.py` wraps the whole sequence. It backs up the recorded turns before anything can overwrite them.

## 7. Running it

```
python game/play.py                      play the slice. wasd to move, e to wait
python architect/architect.py            full run, replay provider, no key
python architect/architect.py --dry-run  reason and stop, write nothing
python architect/architect.py --live     real API calls, needs a key
python tests/test_architect.py           35 assertions
python tests/smoke.py                    five scripted playthroughs
```

Python 3.8 or newer. Nothing to install.

**What a run does on this tree.** The charge feature described in section 1 is already in `game/rex/nemesis.py`, so the game works the moment you open it and `state/BUILT.md` records that run. Running the agent therefore picks up where it left off and builds the next feature, `aggression`, which is the GDD §4 output contract field that the two-tile sprint hangs off. That run is recorded and will verify. A third run selects `sprint_move`, has no recorded turn, and says so rather than guessing. Use `--live` with a key from there.

## 8. Known limits

The GDD says a fight should surface 8 to 10 read lines. A straight run surfaces 2 to 4, because the suppression rule drops repeats and a predictable player keeps triggering the same category. The fix is a richer trigger ladder in `Nemesis._fire`, and it is sitting in `state/NEXT.md` as a gap for the next run rather than something I hand-patched.

The slice is Act 3 only. Acts 1 and 2, the Chronicler and the Gauntlet are not in it, which is deliberate: GDD §8 builds the last level first.

The feature graph in `transcript/` is one model's reading of the GDD. A different run could name the systems differently, and the dependency edges are the part I would check first if the selection ever looked wrong.

The scoring weights are asserted, not derived. They sum to 1.0 and they are printed on every run so the ranking can be argued with.

## 9. Files

```
game/                the capstone slice, standard library only
  play.py            terminal entry point
  rex/               arena, dog, nemesis, reads, loop
  data/              the three DataTables generated by Assignment #4
architect/
  architect.py       orchestration and the printed reasoning trace
  gdd.py             deterministic requirement extraction
  scan.py            ast-based codebase perception, stub detection
  gaps.py            four-level evidence classification
  score.py           utility scoring, four factors
  build.py           prompt assembly, generation, verify, rollback
  memory.py          markdown state
  llm.py             raw urllib provider, live and replay
docs/                the GDD the agent reads
state/               BUILT, FAILED, NEXT, DECISIONS, run_report.json
transcript/          the two recorded model turns, hash-keyed
tests/               35 assertions plus five scripted playthroughs
tools/record_turn.py records a turn answered outside the live path
```